

SELF-STUDY REPORT: CORE C# CONCEPTS & PROBLEM-SOLVING

Course / Subject: C# Programming & .NET Fundamentals

Date: September 5, 2026

1. EXECUTIVE SUMMARY

This self-study report summarizes the practical and theoretical learning outcomes achieved through implementing various fundamental C# tasks. The study covered exception handling, defensive programming, memory management, array structures, null safety, and essential language syntax enhancements.

2. KEY TECHNICAL CONCEPTS & LEARNING OUTCOMES

Topic / Concept	Key Takeaway & Practical Purpose
Exception Handling & finally	Ensures application robustness by gracefully catching runtime errors (e.g., DivideByZeroException). The finally block executes unconditionally to safely release resources.
Defensive Coding & Parsing	Using int.TryParse() prevents FormatException runtime crashes when converting strings. Defensive checks validate method invariants early.
Memory: Boxing vs. Unboxing	Boxing converts value types to reference types on the heap, whereas unboxing extracts them back. Minimizing unnecessary boxing reduces GC overhead and improves performance.
Arrays: Rectangular vs. Jagged	Rectangular arrays (int[,]) allocate continuous memory blocks. Jagged arrays (int[][]) store references to separate sub-arrays, providing flexible row lengths.
Null Safety Mechanisms	Modern C# null features—such as Nullable Types (T?), Null-Coalescing (??), and Null-Propagation (?.)—prevent critical NullReferenceException errors at compile and runtime.
Method Signatures & Parameters	Learned the strict rules for out parameters (mandatory assignment), optional parameters (default placement at method end), and params arrays (flexibility in argument counts).

3. PRACTICAL PROBLEM SOLVING & ALGORITHMS

During the practical implementation phase, several core algorithmic challenges were addressed:

- **Control Flow & Iteration:** Implemented loops for generating custom ranges, multiplication tables up to 12x, filtered even numbers, and basic exponentiation loops.
- **String & Integer Manipulation:** Handled sequence reversal algorithms for text and integers using mathematical extraction (modulo/division) and array operations.
- **Array Distance Algorithms:** Solved element index comparison problems to find the maximum distance between identical values in an array.
- **LINQ & String Parsing:** Applied string.Split() and LINQ's Enumerable.Reverse() to invert sentence word order in concise single-line expressions.

4. CONCLUSION & REFLECTION

Through this comprehensive assignment, I reinforced both theoretical understanding and hands-on coding skills in C#. Mastering defensive programming, memory behavior, and null safety forms a solid foundation for building maintainable, enterprise-ready .NET applications.